

Fireblocks와 국내외 거래소의 블록체인 인덱서 설계·구현 비교 보고서

Executive Summary

공개 자료만 기준으로 보면, “범용 블록체인 데이터 레이어 + 내부 인덱서 플랫폼”을 가장 구체적으로 공개한 곳은 **Coinbase**입니다. Coinbase는 ChainStorage와 ChainNode를 통해 원시 블록 데이터를 로드밸런싱된 노드 클러스터에서 수집하고, 이를 오브젝트 스토리지와 KV 스토어에 저장한 뒤, DynamoDB·S3·Delta Lake·Kafka·Kubernetes·Spark를 조합해 다양한 질의 패턴용 API와 인덱서를 만든다고 설명합니다. 반면 **Fireblocks**는 거래 오케스트레이션·정책·웹훅·영수증 조회·라우팅을 강하게 공개하지만, 고객에게 범용 블록체인 인덱서 제품을 노출하는 방식으로는 설명하지 않습니다. **Binance, Upbit, Bithumb, Coinone**은 대체로 입출금·잔고·주문 관련 REST/WebSocket API를 풍부하게 공개하지만, 그 뒤의 인덱서 백엔드—노드 운영, 큐, DB, 백필, 포크 처리—는 대부분 공개하지 않습니다. **Korbit**는 예외적으로 **Kafka 중심 이벤트 아키텍처와 Temporal 기반 입출금 워크플로**를 기술 블로그에서 비교적 상세히 공개했습니다. ①

설계 관점에서 보면, 공개 사례는 세 부류로 갈립니다. 첫째, **Coinbase**형은 “원시 체인 데이터 레이어”와 “도메인별 파생 인덱서”를 분리합니다. 둘째, **Fireblocks**형은 온체인 데이터 원장을 범용적으로 질의하는 것보다, **트랜잭션 라이프 사이클 제어, 정책 집행, 웹훅, 영수증, 네트워크 라우팅**에 집중합니다. 셋째, **대부분의 거래소**형은 외부에는 “상태 조회 API / 입출금 API / Private WebSocket”만 제공하고, 내부 인덱서는 블랙박스 유지합니다. 이 패턴은 특히 Upbit·Bithumb·Coinone에서 두드러집니다. ②

정합성과 보안에서는 공통적으로 **비동기 처리, 재시도, 멍등성, 재처리 가능성, 이벤트 재전송, 서명 검증, 워크플로 재개**가 핵심입니다. Fireblocks는 웹훅 순서를 보장하지 않으므로 큐 기반 비동기 처리와 빠른 2xx 응답을 권장하고, `externalTxId` 기반 영구 멍등성과 JWKS 서명 검증·IP allowlisting·balance validation을 문서화합니다. Coinbase는 reorg를 “추가/삭제 이벤트의 시퀀스”로 기록하고, Merkle 검증과 node failover를 결합합니다. Korbit는 Temporal의 durable timer·Retry Policy·Replay·중복 방지 정책으로 입금 계류/반환 워크플로를 안정화했다고 밝힙니다. ③

권고안을 한 문장으로 요약하면 이렇습니다. **중소 거래소**는 “모든 체인의 풀 자체 인덱싱”을 목표로 삼기보다, **주요 체인만 직접 통제하고 나머지는 관리형 RPC/스트리밍/오픈 프레임워크를 혼합**하는 편이 비용 대비 효율적입니다. 반대로 **대형 거래소**는 **Coinbase**처럼 원시 블록 저장소와 파생 인덱서를 분리하는 데이터 플레인 이 장기적으로 더 유리합니다. 기관용 서비스는 **Fireblocks·Prime Server-Signer**류의 정책/서명 플레인과 자체 인덱서 플레인을 분리하는 편이 운영 리스크를 줄입니다. ④

판정 기준과 조사 범위

본 보고서는 공식 개발자 문서, 공식 기술 블로그, 공식 공지, 공식 채용/리크루팅 페이지, 공식 오픈소스/문서를 우선 사용했습니다. 사용자가 요청한 항목 가운데 공개 자료로 직접 확인되는 내용은 “확인됨”, 일부 우회적으로만 드러나는 내용은 “부분 확인”, 공식 자료에서 찾을 수 없는 내용은 “미확인”으로 표기했습니다. 특히 **국내 거래소의 경우 공개 범위가 API 표면에 집중**되어 있어, 노드 토폴로지·큐·DB·백필·포크 처리 같은 내부 구현은 업비트·빗썸·코인원에서 대체로 미확인이고, 코빗만 상대적으로 높은 공개 투명성을 보입니다. ⑤

또한 이 보고서의 핵심 구분은 “블록체인 원시 데이터 인덱서”와 “거래소/커스터디 서비스가 외부에 노출하는 API·웹훅·상태 조회 계층” 사이의 차이를 분리해서 보는 데 있습니다. 예를 들어 Coinbase는 둘 다 공개하지만, Fireblocks와 대부분의 거래소는 **외부 문서상 후자만 강하게 드러나고 전자는 내부화**되어 있습니다. ⑥

Fireblocks

Fireblocks의 공개 문서에서 확인되는 것은 **범용 블록체인 인덱서 제품**이라기보다, **다중 체인 자산/트랜잭션 오케스트레이션 계층**입니다. Fireblocks는 `/blockchains` 와 자산 목록 API를 통해 지원 블록체인과 자산 메타데이터를 제공하고, EVM 체인에 대해서는 `transaction receipt` 조회 API를 제공합니다. 또한 네트워크 연결과 라우팅 정책을 API로 노출하며, Webhooks v2를 통해 트랜잭션·오더 등 리소스 이벤트를 제공합니다. 그러나 Coinbase ChainStorage처럼 “원시 블록을 어떻게 보관하고 어떤 DB로 serving하는가”를 문서화한 범용 인덱서 설명은 공식 개발자 문서에서 확인되지 않았습니다. 따라서 **존재 여부/제공 형태**는 “내부 인덱싱 또는 체인 데이터 추적 기능은 존재하나, 공개적으로는 API·웹훅·영수증 조회 형태로 노출되고, 범용 인덱서 제품 설명은 미확인”으로 정리하는 것이 가장 엄밀합니다. 7

아키텍처 관점에서 공개적으로 추론 가능한 부분은 다음과 같습니다. **노드 운영(Full/Archive/RPC)**은 미확인입니다. 다만 Fireblocks는 다중 체인을 지원하고, 네트워크 라우팅 정책과 receipt 조회를 제공하므로 내부적으로 노드/데이터 소스를 관리하거나 외부 제공자를 조합하는 계층이 있음을 시사합니다. **이벤트 수집 방식**은 Webhooks v2 중심이고, **API 레이어**는 자산·블록체인·트랜잭션 상태·영수증·네트워크 연결을 제공합니다. **메시지 큐**는 구체 기술이 미확인이지만, 공식 문서가 “웹훅을 비동기 큐로 처리하라”고 명시하므로 적어도 권장 아키텍처는 큐 기반입니다. **DB·캐시·검색 엔진·분석 스토어**는 전부 미확인입니다. **병렬성** 측면에서 Fireblocks는 같은 vault account에서 같은 블록체인 표준 계열 자산(EVM 계열 등)은 한 번에 하나씩 처리한다고 설명하며, 반대로 이종 체인(예: Bitcoin과 Solana)은 동시에 처리될 수 있습니다. 이는 입출금 오케스트레이션이 단순 무제한 병렬 인덱서가 아니라 **nonce/서명/정책 제약을 반영한 직렬화 레이어**라는 점을 보여줍니다. 8

정합성과 보안은 Fireblocks 공개 자료의 강점입니다. Webhooks v2는 **이벤트 순서를 보장하지 않으며**, 실패 시 장시간 **지수 백오프 재시도**를 수행하고, 최대 **30일 이내 재전송**을 허용합니다. 또한 에러율이 높은 웹훅은 자동 비활성화될 수 있어, Fireblocks는 **빠른 2xx 응답 + 내부 큐 적재**를 권장합니다. 서명 검증은 **JWKS**로 하고, 보안 가이드는 **IP allowlisting·signature verification·balance validation**을 함께 요구합니다. 특히 수신 입금 후 내부 로직을 돌릴 때는 **incoming transaction 웹훅만 믿지 말고 balance update 웹훅과 block height를 교차검증**하라고 권고합니다. 이는 거래소/커스터디의 인덱서 설계에서 매우 중요한 “입금 감지”와 “잔고 반영”의 분리를 잘 보여줍니다. 9

운영·비용 측면에서도 Fireblocks는 명시적으로 역등성과 배칭을 강조합니다. 출금 대량 처리를 위해 `destinations` 배열을 사용하는 **다중 목적지 배치 전송**을 지원하며, 이는 수수료를 줄이는 대신 사용자 체감 지연을 늘릴 수 있다고 설명합니다. 또한 `externalTxId`를 통한 **영구 역등성**을 제공해, 금융성 출금 API에서 중복 요청이 영구적으로 차단되도록 설계했습니다. 요약하면 Fireblocks는 **인덱서 자체보다 트랜잭션 제어·정합성·서명·이벤트 전달 신뢰성**이 중심인 아키텍처를 공개하고 있습니다. 10

해외 거래소

Coinbase는 공개 자료 기준으로 가장 “인덱서다운” 설명을 제시합니다. ChainStorage는 **로드밸런싱된 노드 클러스터**에서 **raw block**을 동시 추출하고, 이를 **blob storage(S3 계열)**와 **key-value storage(DynamoDB 계열)**에 저장한 뒤, **ELT** 방식으로 나중에 변환합니다. 이때 체인 고유 파서와 체인·불문 파서를 지원하지만, 변환은 ingestion 이후로 미룹니다. 데이터 접근은 **batch API**와 **streaming API**로 분리되어 있고, reorg는 블록을 overwrite하지 않고 **추가(+)** / **제거(-)** 이벤트의 **시퀀스**로 적재합니다. Coinbase는 이를 통해 정합성을 유지하면서 CDC류의 downstream 동기화를 가능하게 한다고 설명합니다. 또한 **Merkle hash validation, node failover, archive node 운영 난제**를 직접 언급하며, AWS 상 Ethereum 예시로 **1~2초 freshness**, 실험적으로 **약 1000 blocks/sec**까지 확장 가능하다고 밝혔습니다. 11

그 위의 파생 인덱서 계층도 비교적 자세합니다. ChainNode는 **Temporal workflows**로 orchestration을 하고, **DynamoDB**를 data sink로, **Golang RPC service**를 serving layer로 사용하며, ChainStorage 변경분을 지속 복제해 재인덱싱합니다. Coinbase의 다른 인덱서 글에서는 **S3 parquet, 지난 계산 상태(DynamoDB)**를 소스로 쓰고, **DynamoDB(실시간 질의), S3(대형 payload), Delta Lake(분석), Kafka(알림)**를 sink로 조합한다고 설명합니다.

Serving은 Kubernetes 위의 Go 서비스, 캐시는 watermark와 immutable contract metadata, 계산은 Spark가 담당하며, 배포 시에는 exclusive deployment locks로 동시에 하나의 job만 업데이트되게 합니다. 즉 Coinbase는 원시 데이터 레이어, 파생 계산 레이어, serving 레이어를 분리한 다층형 인덱서를 공개적으로 운영하고 있습니다. ¹²

외부 제공 형태도 분명합니다. Coinbase Developer Platform은 Onchain Data Quickstart에서 SQL 기반 온체인 데이터 조회와 Base용 Node Playground를 함께 제시하고, Onchain Activity Webhooks로 Base 상 스마트컨트랙트 이벤트·토큰 전송·NFT 활동을 실시간 구독하게 합니다. CDP Webhooks는 at-least-once delivery, 최대 60회 재시도, 온체인 데이터 이벤트 <500ms freshness를 내세웁니다. 따라서 Coinbase는 자체 인덱서 보유 + 외부 데이터 API/웹훅 제품화까지 모두 확인되는 사례입니다. ¹³

Binance는 공개 문서상으로는 거래소 지갑/계정 API와 이벤트 스트림은 매우 풍부하지만, 백엔드 블록체인 인덱서 아키텍처는 거의 공개하지 않는 쪽에 가깝습니다. Wallet API는 GET /sapi/v1/capital/config/getall로 코인별 networkList, depositEnable, withdrawEnable를 제공하고, deposit history · withdraw history · deposit address를 제공합니다. Spot User Data Stream은 balance update가 입출금과 계정 간 이체에서 발생한다고 설명하고, REST API는 백엔드 timeout 시 User Data Stream에 상태가 나타났는지 먼저 확인하고 없으면 재조회하라고 명시합니다. WebSocket 문서는 24시간마다 연결 갱신, ping/pong, 단일 연결 최대 1024 streams 같은 운영 제약도 구체적으로 공개합니다. 그러나 노드 운영, 파서/트랜스포머, DB, 캐시, 백필, 포크 처리, RPC 로드밸런싱은 Binance Exchange 공식 개발자 문서에서 미확인입니다. ¹⁴

다만 Binance 조직 차원의 데이터 인프라 단서는 기술 블로그에 있습니다. Risk AI 글에 따르면 Binance는 실시간 파이프라인에서 Flink job, Kafka 기반 ingestion, S3 또는 Elasticsearch sink, 그리고 Kafka/Kinesis/Hive/Snowflake를 조합한 batch+stream 구조를 운용합니다. 또 다른 글은 Kafka의 7일 보존 한계 때문에 S3/Hive 백업을 두고, notebook 환경에서 backfill을 수행한다고 설명합니다. “개발 완료 시점이 예측 불가한 블록체인 입출금 인덱서”를 직접 설명한 것은 아니지만, 대규모 실시간 이벤트를 Kafka/Flink/Hive/S3/ElasticSearch로 처리하는 조직 역량은 공식적으로 확인됩니다. 따라서 Binance의 인덱서에 대해 엄밀하게 말하면 “공개된 거래소 API/실시간 스트림은 확인됨, 내부 인덱서 구성은 미확인, 다만 조직 전체의 대규모 스트리밍 스택은 공식 글로 확인됨”이 정확한 결론입니다. ¹⁵

국내 거래소

국내 거래소는 전반적으로 외부 API 표면은 촘촘하지만 내부 인덱서 구현을 거의 공개하지 않는 경향이 강합니다. 예외는 코빗으로, 코빗은 2025~2026년 공식 기술 블로그를 통해 Kafka·Event Sourcing·Temporal·Go/Rust·gRPC/protobuf·EKS까지 비교적 적극적으로 설명합니다. 반면 업비트·빗썸·코인원은 공식 개발자센터에서 입출금/주문/잔고/웹소켓 기능은 잘 문서화하지만, 노드/큐/DB/백필/포크 처리 내부를 드러내지 않습니다. ¹⁶

업비트는 존재 여부/제공 형태가 매우 분명합니다. 공식 문서는 GET /v1/status/wallet로 입출금 서비스 상태, GET /v1/deposits/chance/coin으로 입금 가능 여부·최소 컨펌 수 (minimum_deposit_confirmations), 입금 주소 생성/목록, 그리고 Private WebSocket myAsset / myOrder를 제공합니다. 다만 업비트는 입출금 상태와 입금 가능 정보가 실시간을 보장하지 않으며 수 분 지연될 수 있다고 명시합니다. Private WebSocket도 myAsset 최초 구독 시 수분간 데이터가 오지 않을 수 있고, Best Practice 문서는 idle timeout 120초, private endpoint 분리, 인증 토큰, 재연결을 구현 요구사항으로 설명합니다. 따라서 업비트는 내부 인덱서 결과를 REST/API와 private stream으로 제공하는 구조로 보이지만, 노드 운영, 파서/트랜스포머, 메시지 큐, DB, 캐시, 백필, 장애복구의 내부 형태는 공식 자료 미확인입니다. ¹⁷

빗썸도 유사합니다. 빗썸 Developer Docs는 REST API와 WebSocket으로 시세·주문·입출금·계정 기능을 제공한다고 명시하고, GET /v1/status/wallet로 입출금 현황과 블록 상태를 조회하게 합니다. 또한 2024년 9월부터 Private WebSocket MyOrder, MyAsset를 지원한다고 공식 공지했습니다. 즉 존재 여부/제공 형태는 “내부 상태를 외부 API와 private stream으로 노출”로 확인되지만, 노드/인덱서 파이프라인·큐·DB·캐시·백필·포크 처리·장

애플북구는 공식 자료 미확인입니다. 빗썸은 공개 표면에서는 업비트와 유사하게 **API 중심·백엔드 비공개형**에 가깝습니다. 18

코인원은 공개 API의 상태 모델이 상대적으로 상세합니다. 가상자산 입출금 내역 API는 `txid`, `confirmations`, `입출금 주소`, `상태값`을 제공하며, 상태는 `DEPOSIT_WAIT`, `DEPOSIT_SUCCESS`, `DEPOSIT_REJECT`, `WITHDRAWAL_REGISTER`, `WITHDRAWAL_WAIT`, `WITHDRAWAL_REFUND_FAIL` 처럼 세밀합니다. 또한 **가상 자산 입금 주소 조회**, **Private WebSocket MYASSET**를 제공해 잔고 변경을 실시간 스트림으로 보냅니다. 이는 코인원이 **온체인 감지 → 상태 모델링 → 계정 반영**을 내부적으로 수행하고 있음을 분명히 보여주지만, 노드/RPC·큐·DB·캐시·포크 처리·백필의 정확한 구현은 공식 문서에 없습니다. 19

다만 코인원의 공식 리크루팅 인터뷰는 백엔드 운영 방식을 일부 드러냅니다. 코인원은 **Kotlin/Spring 기반 MSA 전환**, **AWS Managed Service 활용**, **Replica DB에서 읽은 데이터를 AWS DMS로 실시간 동기화**하고 **Spring Boot Batch 서버로 가공·적재**하는 구조를 설명합니다. 또한 AML 정보를 다루는 **배치 프로그램** 개발 사례를 언급합니다. 이것은 “거래소 백엔드 및 AML 파이프라인” 단서로는 유효하지만, **블록체인 인덱서 전용 메시지 큐·DB·캐시·노드 토폴로지**를 확인할 수는 없으므로 여기 역시 미확인이 적절합니다. 20

코빗은 국내 공개 사례 중 가장 정보량이 많습니다. 공식 API 문서는 REST와 WebSocket을 모두 제공하며, Open API가 **비동기 시스템이라 응답 데이터에 약간의 지연이 있을 수 있다**고 직접 밝힙니다. WebSocket은 Public/Private를 분리하고, Private에서 `myOrder`, `myTrade`, `myAsset`을 제공합니다. 입금/출금 상태는 `pending`, `actionRequired`, `reviewing`, `processing`, `done`, `failed` 등으로 세분화되어 있고, `transactionHash`는 아직 블록체인에 보내지지 않았으면 `null`을 반환한다고 명시합니다. 이 수준의 공식 문서화만으로도 코빗은 **단순 REST 조회가 아니라 명시적인 상태 기계(state machine)**를 외부에 노출하는 거래소로 볼 수 있습니다. 21

기술 블로그는 더 직접적입니다. 코빗은 차세대 거래 시스템에서 **Kafka 중심 비동기 이벤트 아키텍처**와 **Event Sourcing**을 사용한다고 설명하고, 주문 서비스가 이벤트를 발행하면 체결 엔진이 소비하고, 다시 잔고·시세 서비스가 이를 소비해 상태를 갱신한다고 말합니다. 언어 선택도 공개되어 있는데, **주문/Open API는 Go**, **체결 엔진/시세 서비스는 Rust**, 서비스 간 통신은 **gRPC 또는 Kafka에 실은 Protocol Buffers**, 런타임은 **AWS EKS**입니다. 이는 “**핵심 엔진은 저지연·메모리 안정성 우선, 애플리케이션 계층은 생산성 우선**”이라는 대형 거래소류 분리 전략과 매우 유사합니다. 22

입출금/블록체인 워크플로 공개 수준은 더 높습니다. 코빗 플랫폼 팀은 **입금 감지 → 외부 AML 솔루션(Chainalysis 등) 스크리닝 → 잔고 반영 또는 계류 → 반환 확정 시 자산 이동·블록체인 출금** 흐름을 설명하며, 이를 **Temporal**로 구현했다고 공개했습니다. `durable timer`(`Workflow.sleep`), Activity별 Retry Policy, 장애 후 Replay, 멍등성, `ALLOW_DUPLICATE_FAILED_ONLY`, `continueAsNew`, 대시보드 기반 추적 등은 **거래소 입출금 인덱싱/처리 계층이 단순 cron+table이 아니라 durable workflow engine 위에 올라가고 있음**을 보여줍니다. 또 Blockchain Tech 팀의 Chronos 글은 **입출금 중단/재개 예약**, **Slack Fail-safe 알림**, 향후 **노드 상태·수수료 변화·업그레이드 실시간 모니터링**, **수수료 지갑 자동 충전**까지 계획한다고 밝힙니다. 국내 거래소 공개 자료 중에서는 코빗이 **정합성·운영자 동화·장애복구 측면에서 가장 성숙한 설계 철학**을 외부에 보여주고 있습니다. 23

비교와 권장 아키텍처

아래 표는 조직별로 공개 자료에서 확인된 핵심 설계 선택을 압축한 것입니다. “미확인”은 공개 문서상 확인되지 않았다는 뜻이지, 실제로 존재하지 않는다는 뜻은 아닙니다.

조직	공개 확인 수준	인덱서 존재 여부 및 제공 형태	공개된 핵심 설계 선택	정합성·보안·복구 포인트	미확인 영역	근거
Fireblocks	부분 확인	내부 체인 추적 + API/웹훅/영수증 조회 제공. 범용 고객용 인덱서 제품 설명은 미확인	블록체인/자산 목록 API, EVM receipt 조회, network routing, Webhooks v2, multi-destination batching	순서 비보장, exponential backoff 재시도, 30일 재전송, JWKS 서명 검증, IP allowlisting, balance validation, externalTxId 영구 멍등성	노드 운영, 큐 기술명, DB, 캐시, 검색/분석 스토어, 백필 전략	Fireblocks 공식 문서 24
Coinbase	확인 됨	자체 인덱서 플랫폼 + 외부 CDP 데이터/API 제품화	로드밸런싱 노드 클러스터, ELT, S3형 blob + DynamoDB형 KV, chain-native parser, ChainNode/Temporal/Go RPC, DynamoDB/S3/Delta Lake/Kafka, K8s, Spark, 캐시 (watermark·metadata)	reorg를 add/remove event sequence로 기록, Merkle 검증, node failover, 배포 lock, streaming/batch 병행	일부 세부 캐시/서빙 스키마, 체인별 노드 수량	Coinbase 공식 가이드, 블로그, 발자 문서 25
Binance	부분 확인	Wallet API + User Data Stream + 상태/이력 API 제공. 내부 인덱서 구현은 미확인	coin/network metadata, deposit/withdraw history, balance update 실시간 이벤트, 운영 제한이 명시된 WebSocket. 조직 차원 기술 블로그에는 Flink/Kafka/Kinesis/Hive/Snowflake/S3/ElasticSearch 기반 스트림 파이프라인 공개	REST timeout 후 stream 기반 재조회 권고, batch+stream 결합, historical backfill(S3/Hive)	블록체인 노드 운영, 파서, 포크 처리, DB, 캐시, 인덱싱 파티셔닝	Binance 공식 가이드, 자문서, 솔블로 26
Upbit	부분 확인	입출금 상태/입금 가능 정보/입금 주소 + Private WebSocket 제공	status/wallet, deposits/chance/coin, myAsset, myOrder, private endpoint, reconnect/keepalive 지침	상태 API와 입금 가능 정보가 수분 지연 가능, minimum_deposit_confirmations 제공, WebSocket idle timeout 120초, 최초 myAsset 구독 지연 가능	노드, 큐, DB, 캐시, 포크 처리, 백필, 장애복구 내부 구현	업비트 공식 개발 문서 1

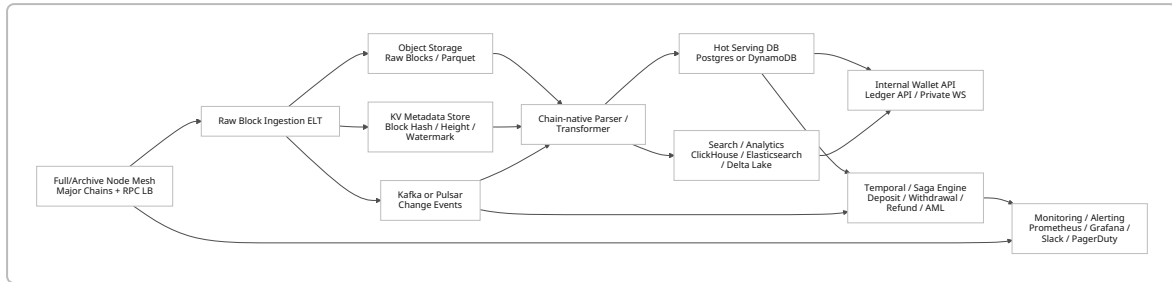
조직	공개 확 인 수 준	인덱서 존재 여부 및 제공 형태	공개된 핵심 설계 선택	정합성·보안·복구 포인트	미확인 영역	근거
Bithumb	부 분 확 인	입출금 상태/ 입출금 API + Private WebSocket 제공	REST+WebSocket, wallet/block status 조 회, MyOrder · MyAsset private websocket	외부 API 표면은 풍부하나 내부 정합성 메커 니즘은 비공개	노드, 큐, DB, 캐시, 백 필, 포크 처리, 장 애복구	빗썸 공 개발자 서/공지 18
Coinone	부 분 확 인	입출금 이력/ 입금 주소/ Private WebSocket 제공	txid, confirmations, 세 밀한 deposit/ withdraw/refund/ reject status, MYASSET 실시간 스트 림. 백엔드 인터뷰상 Kotlin/Spring MSA, Replica DB + AWS DMS + Spring Batch, AML batch	상태 모델이 세밀하고 환급/거절 상태까지 노출	인덱서 전용 노 드, 큐, DB, 포 크 처리, RPC 로 드밸런 싱	코인원 식 개발 문서/공 리크루 인터뷰 27
Korbit	확 인 됨	내부 이벤트/ 워크플로 기 반 처리 + 외 부 REST/ WebSocket 제공	Kafka event bus, Event Sourcing, Go(Open API/주문) + Rust(체결/ 시세), gRPC/protobuf, AWS EKS, Temporal 기 반 입금 계류·반환·운영자 동화	durable timer, Retry Policy, Replay, 중 복 방지, continueAsNew, Slack fail- safe, 상태기계 중심 입출금 처리	인덱서 전용 저 장소 종 류, 체인 별 full/ archive 토폴로 지	코빗 공 API/기 블로그 28

비교표가 보여주는 핵심은, **거래소 인덱서는 보통 하나의 모놀리식 서비스가 아니라 세 개의 분리된 평면**이라는 점입니다. 첫째, **원시 체인 데이터 수집 평면**이 있고, 둘째, 이를 특정 조회 패턴에 맞게 바꾸는 **파생 인덱서/서빙 평면**이 있으며, 셋째, 입금·출금·환급·AML·승인·잔고 반영을 책임지는 **원장/워크플로 평면**이 있습니다. Coinbase는 이 세 평면을 가장 분명하게 공개했고, Fireblocks는 워크플로/정책 평면을, Korbit는 거래소 이벤트/워크플로 평면을 강하게 드러냈습니다. 업비트·빗썸·코인원·Binance는 외부에서 주로 서빙 평면만 보이는 구조입니다. 29

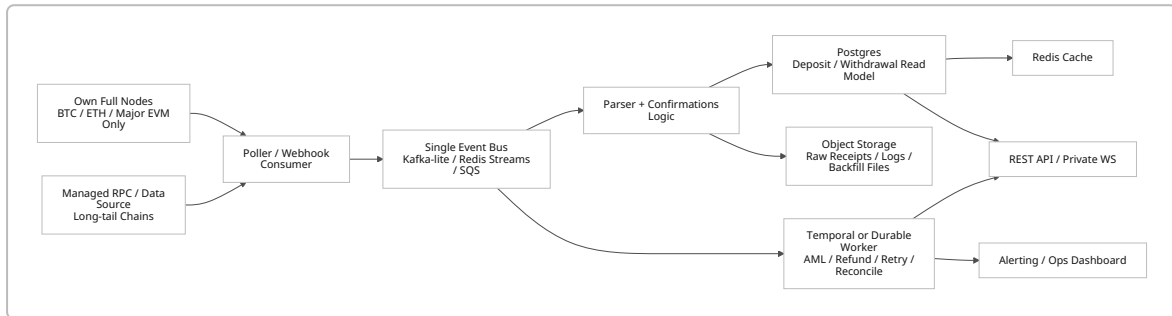
오픈 프레임워크 활용 장단점도 분명합니다. **The Graph**는 Subgraphs·Substreams와 GraphQL 질의 레이어를 기본 제공해 “컨트랙트 이벤트 → 읽기 API”를 가장 빠르게 만드는 쪽에 강하고, **SQD**는 arbitrary block ranges·streaming·reorg handling과 any-DB/파일 기반 데이터셋을 전면에 내세워 멀티체인 장기보존과 장거리 백필에 유리합니다. **Envio HyperIndex/HyperSync**는 멀티체인·GraphQL·내장 reorg support를 제공하고, HyperSync는 전통적 RPC 대비 훨씬 빠른 동기화를 지향하며 Postgres를 기본 저장소로 두고 ClickHouse 분석 저장을 붙이는 방향까지 보여줍니다. 다만 공개 사례를 종합하면, 이런 프레임워크는 **읽기용 색인 계층**으로는 좋지만, 거래소가 반드시 필요한 **승인·AML/FDS·출금 환급·내부 원장 정합성**까지 대체하지는 못합니다. 그래서 실제 거래소 설계에서는 **인덱서 프레임워크와 워크플로 엔진을 분리**하는 편이 안전합니다. 30

대형 거래소 권장안은 Coinbase의 raw-data lake + multi-sink serving, Korbit의 Kafka/Temporal식 워크플로, Fireblocks의 웹훅 보안·역등성 패턴을 결합하는 형태가 가장 합리적입니다. 핵심은 **(원시 블록을 버리지 말고 저장할**

것) / (도메인별 파생 인덱서를 분리할 것) / (입출금 상태 전이를 durable workflow에 올릴 것)입니다. Coinbase가 raw block + KV + streaming API + multi-sink 아키텍처를 공개했고, Korbit가 워크플로/운영자동화를 공개했으며, Fireblocks가 역등성과 webhook 신뢰성을 강조한다는 점이 이 권고의 근거입니다. 31



중소 거래소 권장안은 더 단순해야 합니다. 모든 체인마다 archive node를 운영하는 대신, 거래량 상위 소수 체인만 직접 운영하고, 나머지 체인은 관리형 RPC 또는 SQD/HyperIndex류 데이터 소스를 쓰는 혼합형이 현실적입니다. Coinbase가 archive node 유지의 어려움과 node provider 비용을 지적했고, SQD와 HyperSync가 long-range extraction · reorg handling · 고속 동기화를 전면에 내세우는 만큼, 중소 거래소는 원시 블록 보관 + Postgres 중심 서빙 + Redis 캐시 + 단일 워크플로 엔진 조합으로 시작하는 편이 비용과 정합성의 균형이 좋습니다. 32



유형별 추천안을 정리하면 아래와 같습니다.

대상	추천 구성	왜 이 구성이 적합한가	주의할 점	근거
중소 거래소	상위 체인만 자가 Full/Archive, 롱테일은 관리형 RPC/색인 소스, Postgres + Redis, 원시 로그/receipt는 오브젝트 스토리지 보관, 하나의 durable workflow 엔진으로 입출금·환급·재시도 처리	초기 비용과 운영 인력을 줄이면서도 재처리·백필·감사 추적이 가능함	관리형 소스 장애 시 fallback 경로 필요, 상태 API와 원장 반영을 분리해야 함	Coinbase의 archive node/비용 언급, SQD/HyperSync의 long-range/reorg 기능, Fireblocks·Korbit의 workflow/역등성 사례 33

대상	추천 구성	왜 이 구성이 적합한가	주의할 점	근거
대형 거래소	Coinbase형 raw block lake + KV metadata + streaming CDC + 파생 인덱서 다중 sink(DynamoDB/Postgres/ClickHouse/Delta Lake) + Kafka + K8s/Spark + Temporal	체인 추가와 백필, 다양한 질의 패턴, 분석/탐지/정산을 동시에 만족시키기 쉬움	초기 구축비와 데이터 거버넌스 비용이 큼. 서비스별 DB 선택을 엄격히 분리해야 함	Coinbase 공식 사례, Korbit의 이벤트/워크플로 공개 사례 ³⁴
기관용 서비스	Fireblocks류의 정책·서명·웹훅 플레인 또는 Prime Server-Signer류와, 내부 정산/인덱서 플레인을 분리. 인덱서는 체인별 최소 read model 중심으로 운영	기관 서비스는 범용 조회보다 승인·정책·서명·감사·역등성이 더 중요함	서명 평면과 데이터 평면을 혼합하면 장애 전파 환경이 커짐	Fireblocks webhook/idempotency/security, Coinbase Prime server-signer 문서 ³⁵

최종 결론은 명확합니다. 거래소 인덱서는 “RPC를 읽어서 DB에 넣는 프로그램”이 아니라, 거래소 원장과 고객 자산을 안전하게 연결하는 정산 시스템의 일부입니다. 그래서 좋은 설계는 단순히 빠른 인덱싱보다 **재처리 가능성, reorg 안전성, 역등성, 운영 자동화, 이벤트 재전송, 상태 전이의 가시성**을 우선합니다. 공개 자료 기준으로는 Coinbase가 데이터 플레인, Fireblocks가 정책/트랜잭션 플레인, Korbit가 국내 거래소 워크플로 플레인을 가장 잘 보여주고 있습니다. Upbit·Bithumb·Coinone·Binance는 외부 API는 풍부하지만 내부 인덱서는 전략적으로 비공개에 가깝습니다. 따라서 설계 선택의 출발점은 “모든 것을 직접 만들 것인가”가 아니라, **어떤 평면을 직접 소유해야 사업 리스크를 통제할 수 있는가**를 먼저 정하는 것입니다. ³⁶

¹ ² ⁴ ⁶ ¹¹ ²⁵ ²⁹ ³¹ ³² ³³ ³⁴ ³⁶ Part 1: ChainStorage: The Enterprise Blockchain data availability layer

<https://www.coinbase.com/blog/part-1-chainstorage-the-enterprise-blockchain-data-availability-layer>

³ ⁹ <https://developers.fireblocks.com/reference/webhooks-gettingstarted-responsesretries>
<https://developers.fireblocks.com/reference/webhooks-gettingstarted-responsesretries>

⁵ ¹⁶ ¹⁷ <https://docs.upbit.com/kr/reference/get-service-status>
<https://docs.upbit.com/kr/reference/get-service-status>

⁷ ²⁴ <https://developers.fireblocks.com/api-reference/blockchains-%26-assets/list-blockchains>
<https://developers.fireblocks.com/api-reference/blockchains-%26-assets/list-blockchains>

⁸ <https://developers.fireblocks.com/reference/statuses>
<https://developers.fireblocks.com/reference/statuses>

¹⁰ <https://developers.fireblocks.com/docs/manage-withdrawals-at-scale>
<https://developers.fireblocks.com/docs/manage-withdrawals-at-scale>

¹² Part 3: From Block to APIs: Building Indexers on ChainStack
<https://www.coinbase.com/blog/part-3-from-block-to-apis-building-indexers-on-chainstack>

- 13 <https://docs.cdp.coinbase.com/data/get-started/quickstart>
<https://docs.cdp.coinbase.com/data/get-started/quickstart>
- 14 26 <https://developers.binance.com/docs/wallet/capital>
<https://developers.binance.com/docs/wallet/capital>
- 15 <https://www.binance.com/en/blog/tech/7972341655591522254>
<https://www.binance.com/en/blog/tech/7972341655591522254>
- 18 <https://apidocs.bithumb.com/docs/%EB%B9%97%EC%8D%B8-developer-docs>
<https://apidocs.bithumb.com/docs/%EB%B9%97%EC%8D%B8-developer-docs>
- 19 27 <https://docs.coinone.co.kr/reference/coin-transaction-history>
<https://docs.coinone.co.kr/reference/coin-transaction-history>
- 20 채용 홈페이지 - 코인원
https://recruit.coinonecorp.com/blog3?utm_source=chatgpt.com
- 21 28 <https://docs.korbit.co.kr/>
<https://docs.korbit.co.kr/>
- 22 완전히 새롭게 구축한 코빗의 가상자산 거래 시스템
<https://tech.korbit.co.kr/posts/system-architecture-design-of-cryptocurrency-exchange/>
- 23 플랫폼 팀의 Temporal 사용기: 입금 계류 및 반환 작업 편
<https://tech.korbit.co.kr/posts/building-deposit-pending-with-temporal/>
- 30 <https://thegraph.com/docs/en/>
<https://thegraph.com/docs/en/>
- 35 <https://developers.fireblocks.com/reference/validating-webhooks>
<https://developers.fireblocks.com/reference/validating-webhooks>